

Programmazione Modulare

Al giorno d'oggi, i progetti software:

- sono complessi e composti da moltissime righe di codice,
- si basano sul riuso di funzionalità (librerie) scritte da altri programmatori.

Avere quindi il codice di un programma scritto su un unico file è impraticabile.

Una piccola modifica in un unico punto non può obbligare ad una ricompilazione di tutto il progetto.

L'idea è raggruppare le funzioni che si occupano di una specifica funzionalità in un unico file sorgente, chiamato **Modulo**, composto da:

- **Interfaccia - Header File (.h):**
 - è **composto dalla lista dei Prototipi delle Funzioni e dei Tipi di Dato** del modulo
 - è inclusa con la direttiva `#include` in ogni modulo/applicazione che lo usa
 - non deve essere mai compilato
- **Implementazione (.c):**
 - **in cui viene realizzato il Codice che Implementa le funzioni contenute nell'Header**
 - non contiene mai la funzione `main()`

L'Applicazione:

- utilizza uno o più moduli includendo i corrispondenti header file,
- contiene la funzione `main()`,
- è compilato e linkato con il file oggetto dei moduli

```
#include "operazioni.h"
```

```
int main(void) {  
    int i = 3;  
    int j = 5;  
  
    printf("i+j = %d\n", somma(i,j) );  
    printf("i*j = %d\n", moltiplica(i,j) );  
}
```

applicazione.c

```
int moltiplica(int primo, int secondo);  
int somma(int primo, int secondo);
```

operazioni.h

```
#include "operazioni.h"
```

```
int moltiplica(int primo, int secondo) {  
    return primo*secondo;  
}
```

```
int somma(int primo, int secondo) {  
    return primo+secondo;  
}
```

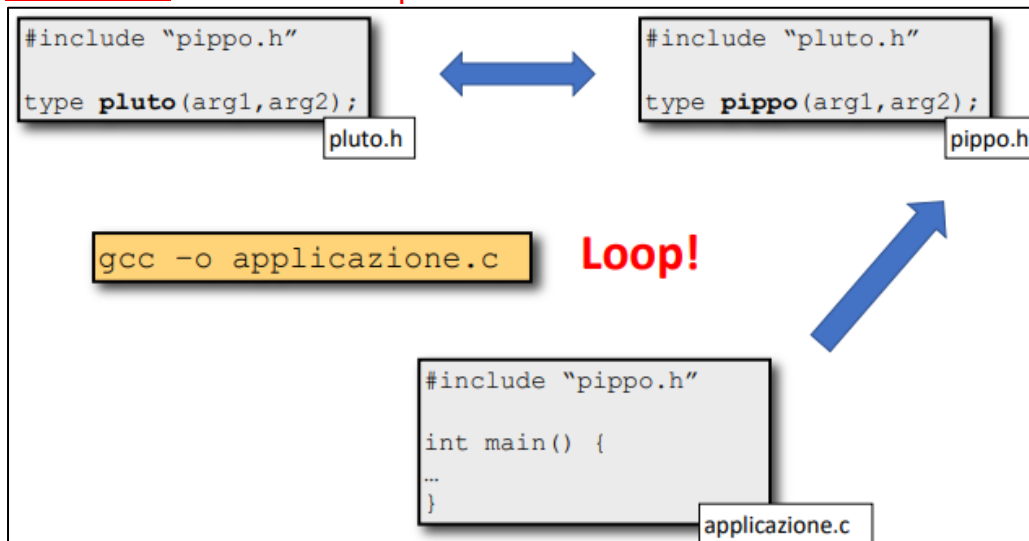
operazioni.c

ATTENZIONE: esiste una dipendenza fra i file .h e il file .c che li include:
se un file di intestazione cambia, allora i moduli dipendenti devono essere ricompilati!

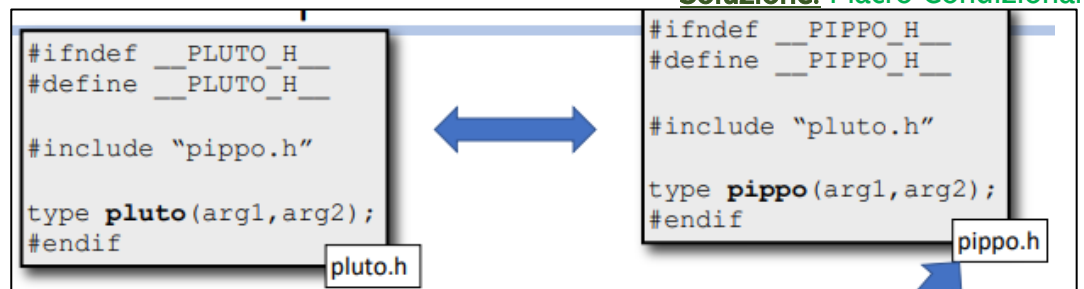
Problema 1: Efficienza

- Se il progetto consiste di centinaia di files, può tuttavia essere utile la compilazione separata. Come determinare quali moduli ricompilare?
 - Dimenticando di ricompilarne qualcuno, può capitare che il linker generi errori o, peggio, se l'errore non sarà segnalato preventivamente, errori a runtime.
 - **Soluzione:** Utility Make (...)

Problema 2: Ciclicità delle Dipendenze



Soluzione: Macro Condizionali



Utility Make

È uno strumento che può essere utilizzato per automatizzare il processo di compilazione:

- Si basa sull'utilizzo di un "makefile" che descrive le dipendenze presenti nel progetto;
 - è quindi possibile utilizzare il comando make che si avvale della marcatura temporale dei files e ricompila i target file che sono più vecchi dei sorgenti.

Make: Regole Implicite

Il comando `gcc test.c -o test` è equivalente al comando `make test` dove «test» è il target:

- se non vi sono regole esplicite definite (...), **make tenta di creare un eseguibile** cercando il file «test.c» oppure «test.o»
- è possibile anche solo la compilazione senza linking con il comando `make test.o`

Make: Regole Esplicite

Il makefile elenca, in un file di testo, un insieme di target rules: regole per la compilazione e l'istruzione da eseguire per compilare a partire dai sorgenti.

Le regole esplicite hanno una forma del tipo:

```
nome_target: file1.o file2.o filen.o
    gcc -o nome_target file1.o file2.o filen.o
file1.o: file1.c file1.h
    gcc -c file1.c
...
filen.o: filen.c filen.h
    gcc -c filen.c
clean:
    rm -f *.o
```

La regola *clean* permette di rimuovere tutti i file oggetto

`make nome_target`

Nell'invocare make da linea di comando è possibile specificare quale target compilare (es: `make file1.o`)

- Se il comando make viene invocato senza opzioni dalla linea di comando, utilizza il target di default: il primo specificato nel makefile.
- Infine, con il comando `make nome-target -f miomakefile` è possibile specificare il nome di un makefile (se diverso da "makefile")

Make: Variabili D'Ambiente

Ogni processo ha associato un array di stringhe che contiene l'elenco delle variabili di ambiente

- Ogni elemento dell'array ha la forma `nome=valore` (esempi: HOME, LOGNAME, PATH, ...)
- Ogni volta che un processo viene creato, eredita le variabili di ambiente del processo creatore
- All'interno di un programma C:
 - la lista delle variabili di ambiente può essere consultata con la var. globale `char **environ`
 - La funzione `char * getenv (const char *name)` restituisce la stringa della variabile di ambiente passata come parametro

In generale, si possono definire alcune variabili di ambiente (anche internamente al makefile):

- CC - Il comando da usare per il compilatore
- CFLAGS - I flags da usare durante la compilazione (es: `export CFLAGS="-std=c89 -pedantic"`)
- LDFLAGS - I flags per la fase di linking (es: `export LDFLAGS= "-lm"`)

Con il comando `printenv` è possibile visualizzare l'elenco di tutte le variabili d'ambiente definite

Con il comando `echo $nome-var` è possibile visualizzare il valore della var. d'ambiente "nome-var"

Ad esempio: `export PIPPO="10"`

`echo $PIPPPO -> stamperà il valore: 10`